研艺士
YANZHISHI

提高篇 （2）——数据结构专题

主讲人：灵灵老师

计算机考研

目录

1. 二叉树的相关概念
2. 二叉树的遍历
3. 二叉排序树
4. 哈夫曼树

研学士 YANZHISHI

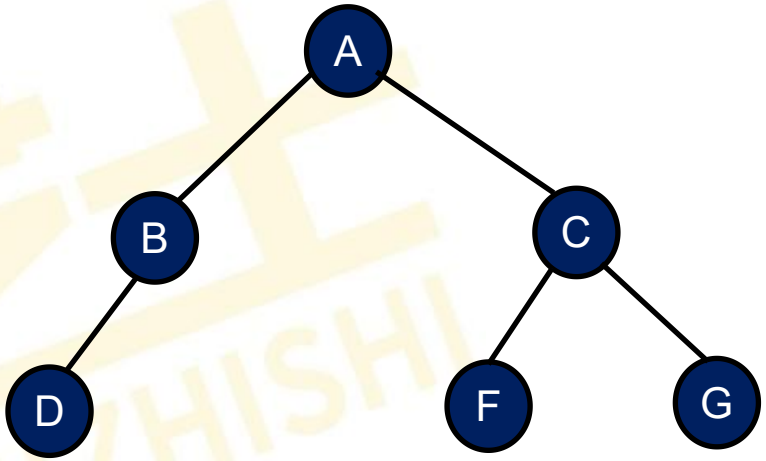
二叉树的相关概念

二叉树的定义

二叉树是n(n≥0)个结点的有限集合。n=0则为空树，否则：

(1)若n=1，只有一个根结点；

(2)若n>1，除根结点外，其余的结点可以划分为两个互不相交的子集T_1和T_2，分别称之为**左子树和右子树**，并且左子树和右子树又都是二叉树。 **二叉树的定义是递归定义。**



二叉树的相关概念

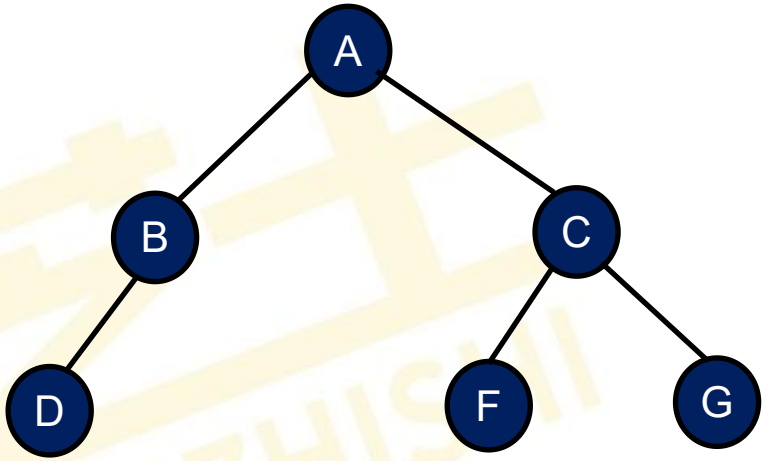
基本术语

孩子结点：某个结点的直接后继结点，在二叉树中，每个结点最多有两个孩子结点。

叶子结点：又称终端结点，没有任何子树（没有任何孩子结点）的结点。

结点的度：结点拥有的子树（或孩子结点）的数量。

树的深度：树中结点的最大层数，又称树的高度。根结点在第1层。



二叉树的相关概念

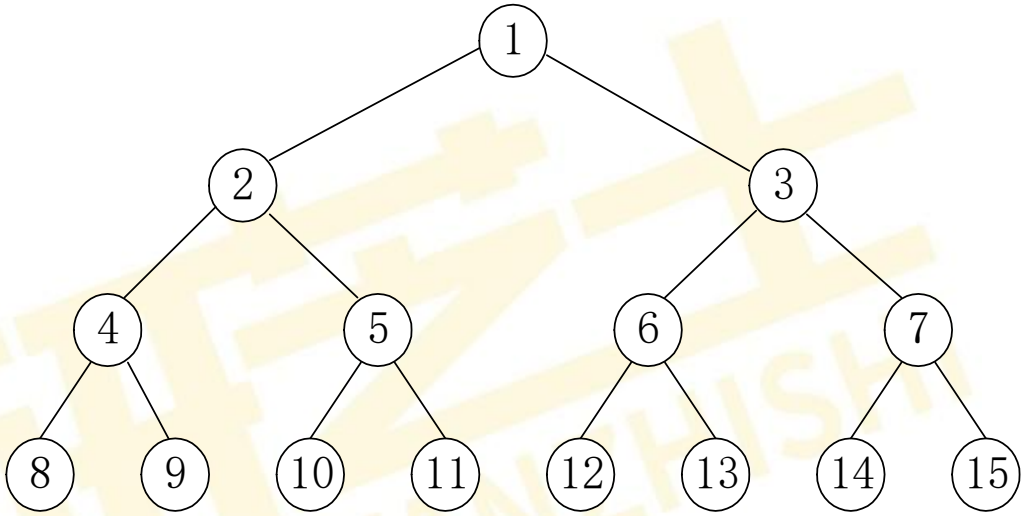
特殊二叉树

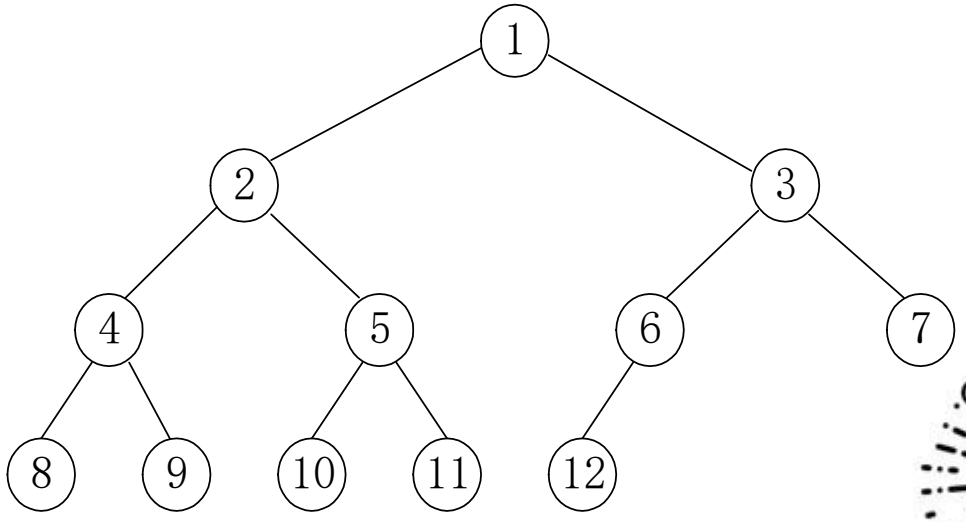
- **满二叉树：**每层结点均满，叶子结点都在最下一层。深度为k，第i层2^{i-1}个结点。

  结点总数：$2^k-1=2^0+2^1+2^2+\cdots+2^{k-1}$。

- **完全二叉树：**只有最下面一层结点不满，且最下层的叶子结点左边连续。

  结点编号与同样深度的满二叉树中结点编号一一对应。编号从上到下，从左到右。





二叉树的相关概念

二叉树的性质

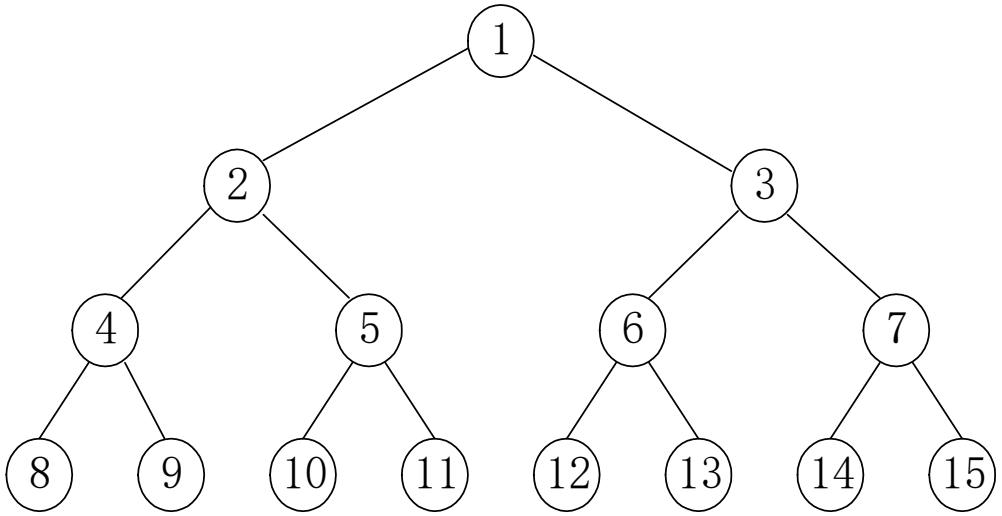
性质1：二叉树非空，**第i层最多有2^{i-1}个结点**($i \geq 1$)。

性质2：深度为k的二叉树，**最多有2^k-1个结点**。

性质3：对于任何一棵二叉树，若叶子结点个数为n_0，度为2的结点个数为n_2，则有$n_0 = n_2+1$。

性质4：具有n个结点的**完全二叉树，深度必为**$\lfloor \log_2 n \rfloor +1$。

性质5：完全二叉树的编号从1开始，从上至下、从左至右。假定第i个结点为当前结点：左孩子结点的编号为2i，右孩子结点的编号为i/2。

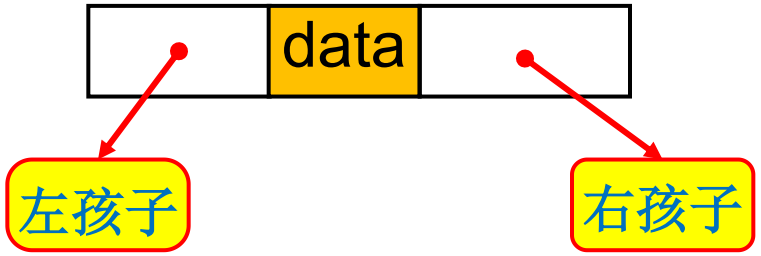


二叉树的链式存储

二叉链表

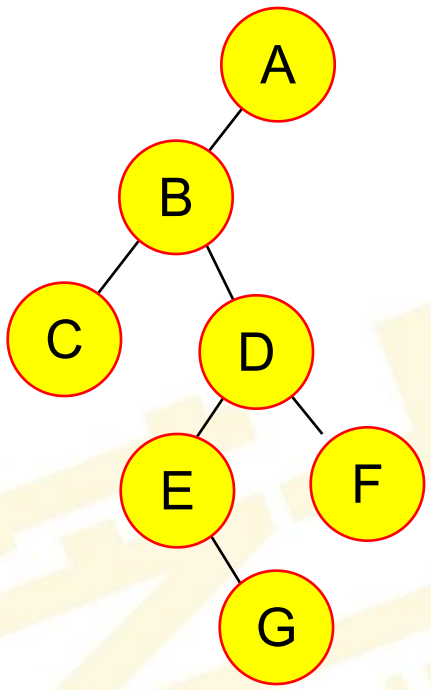
数据域，左指针域(指向左孩子)，右指针域(指向右孩子)。

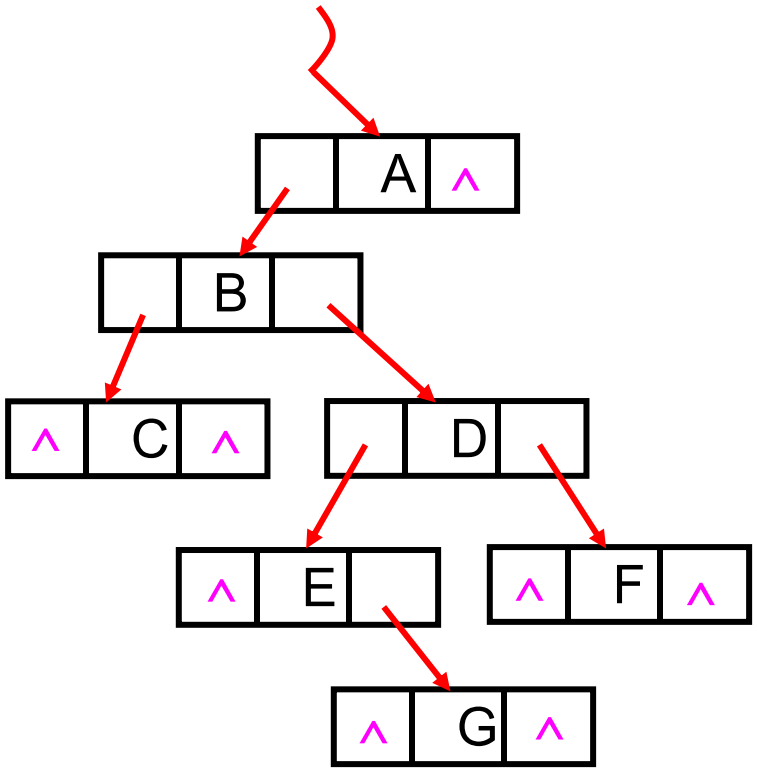
```
typedef struct  BiNode
{   ElemType  data ;
    struct  BiNode  *lchild , *rchild ;   //递归定义
}BiNode ;
```



二叉树的相关概念

链式存储结构示例：





二叉树的相关概念

[3537. 树查找_AcWing题库]

输入格式

第一行包含整数 n。

第二行包含 n 个整数，表示该二叉树的层序遍历序列。

第三行包含整数 k。

输出格式

共一行，按照从左到右的顺序输出该树第 k 层的全部结点编号。

数与数之间用单个空格隔开。

若无该层结点，则输出 `EMPTY`。

数据范围

$1 \le n \le 1000$，
$1 \le k \le 20$

输入样例：

```
4
1 2 3 4
2
```

输出样例：

```
2 3
```

研艺士 YANZHISHI

二叉树的遍历

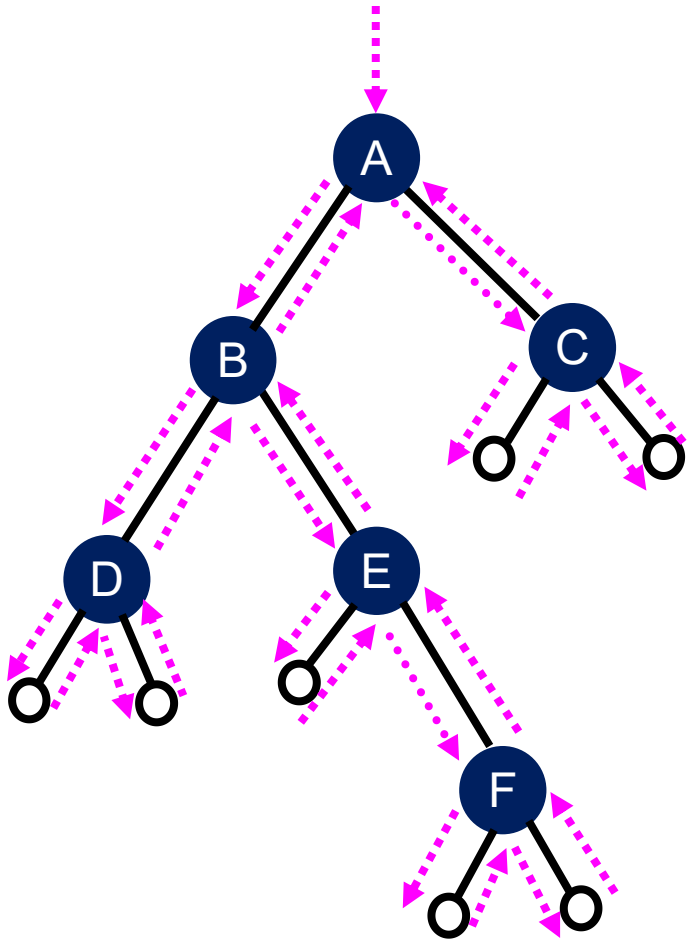
二叉树遍历算法的基本思想

按照访问根结点的顺序：

先(根)序遍历：根结点→左子树→右子树

中(根)序遍历：左子树→根结点→右子树

后(根)序遍历：左子树→右子树→根结点



虚线代表遍历路径，○代表空子树NULL。从上到下，从左到右，在遍历过程中，每个结点经过3次

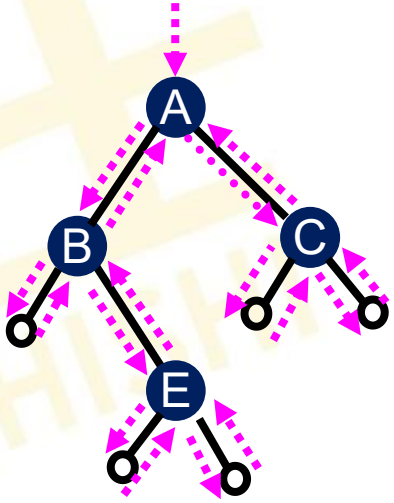
二叉树的遍历

二叉树遍历的实现

先序遍历算法:

```
void PreOrder (BiTree T)
{   if(T)
    {   cout<<T->data;      //先序访问根结点
        PreOrder(T->lchild);  //递归遍历左子树
        PreOrder(T->rchild); //递归遍历右子树
    }
} //若T==NULL，遇此括号，函数自动结束并返回
```

递归算法，系统建栈，
自动入栈，自动出栈。
非递归算法，用户建栈。




```
typedef struct  BiNode
{   ElemType  data ;
    struct  BiNode  *lchild , *rchild ;   //递归定义
}BiNode ,   *BiTree;
```

二叉树的遍历

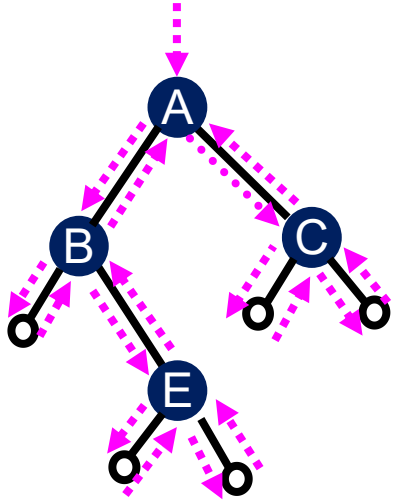
中序遍历算法：

```
void  InOrderTraverse(BiTree T )
{    if(T)
     {    InOrderTraverse(T->lchild);
          cout<<T->data;
          InOrderTraverse(T->rchild);
     }
 }  //若T==NULL，遇此括号，函数自动返回
```

后序遍历算法：

```
 void  PostOrderTraverse(BiTree T )
 {    if(T)
      {    PostOrderTraverse(T->lchild);
           PostOrderTraverse(T->rchild);
           cout<<T->data;
      }
 } //若T==NULL，遇此括号，函数自动返回
```

```
int TreeDepth(BiTree T)    //计算二叉树T的深度
 {    if(T==NULL)     return 0;
      else
      {  int m=TreeDepth(T->lchild);  //左子树的深度
         int n=TreeDepth(T->rchild);  //右子树的深度
         if(m>n) return  (m+1);   //深度为较大者加1
         else    return  (n+1);
      }
 }
```



二叉树的遍历

[3384. 二叉树遍历_AcWing题库]

```
abc##de#g##f###
c b e g d f a
```

编写一个程序，读入用户输入的一串先序遍历字符串，根据此字符串建立一个二叉树（以指针方式存储）。

例如如下的先序遍历字符串： `abc##de#g##f###` 其中 `#` 表示的是空格，空格字符代表空树。

建立起此二叉树以后，再对二叉树进行中序遍历，输出遍历结果。

输入格式

共一行，包含一个字符串，表示先序遍历字符串。

输出格式

共一行，输出将输入字符串建立二叉树后中序遍历的序列，字符之间用空格隔开。

注意，输出中不用包含 `#` 。

数据范围

输入字符串长度不超过 100，且只包含小写字母和 `#` 。

输入样例：

```
abc##de#g##f###
```

输出样例：

```
c b e g d f a
```

研学士 YANZHISHI

二叉树的遍历

KY212二叉树遍历_牛客网 (nowcoder.com)

输入描述：

两个字符串，其长度n均小于等于26。 第一行为前序遍历，第二行为中序遍历。 二叉树中的结点名称以大写字母表示：A，B，C....最多26个结点。

输出描述：

输入样例可能有多组，对于每组测试样例， 输出一行，为后序遍历的字符串。

示例1

```
输入：ABC
      BAC
      FDXEAG
      XDEFAG
输出：BCA
      XEDGAF
```

复制 复制

二叉排序树

二叉排序树的定义

二叉排序树（Binary Sort Tree），又称二叉查找树（Binary Search Tree），亦称二叉搜索树。

二叉排序树或者是一棵空树，或者是具有下列性质的二叉树：

- 左子树上所有结点的值均小于其根结点的值
- 右子树上所有结点的值均大于其根结点的值

特点：按中序遍历得到的序列是一个从小到大的有序序列

研之士 YANZHISHI

二叉排序树

KY207 二叉排序树_牛客网 (nowcoder.com)

输入描述：

输入包含多组测试数据，每组测试数据两行。 第一行，一个数字N（N<=100），表示待插入的节点数。 第二行，N个互不相同的正整数，表示要顺序插入节点的关键字值，这些值不超过10^8。

输出描述：

输出共N行，每次插入节点后，该节点对应的父亲节点的关键字值。

示例1

输入：5 复制
2 5 1 3 4

输出：-1 复制
2
2
5
3

研究生 YANZHISHI

哈夫曼树

哈夫曼树的定义

哈夫曼树（Huffman Tree），又称霍夫曼树或最优二叉树。给定N个权值作为N个叶子结点，构造一棵二叉树，若该树的带权路径长度（Weighted Path Length，WPL）达到最小，则称这样的二叉树为最优二叉树，也称为哈夫曼树。

带权路径长度是指树中所有叶子结点的权值乘上其到根结点的路径长度之和。

哈夫曼树构造步骤

① 排序：根据给定的权值（通常代表字符出现的频率），对叶子结点进行排序。

② 合并：选择权值最小的两个结点作为左右子树，合并成一个新的父结点，其权值为两个子结点权值之和。

③ 重复：重复上述排序和合并步骤，直到所有结点都合并成一棵完整的哈夫曼树。

<priority_queue>

priority_queue 称为优先队列。在优先队列中，队首元素一定是当前队列中优先级最高的那一个。

1. priority_queue 的定义

要使用优先队列，应先添加头文件#include <queue>，并在头文件下面加上“using namespace std;”，然后就可以使用了。

其定义的写法和其他 STL 容器相同，typename 可以是任意基本数据类型或容器：

```
priority_queue< typename > name;
```

2. priority_queue 容器内元素的访问

和队列不一样的是，优先队列没有 front()函数与 back()函数，而只能通过 top()函数来访问队首元素（也可以称为堆顶元素），也就是优先级最高的元素。

YANZHISH

3.priority_queue 常用函数

(1) top () ：获取队首元素。

(2) push()：push(x)将令x入队。

(3) pop()：移除队首元素。

(4) empty()：检查队列是否为空，返回布尔值。

(5) size()：返回队列中元素的个数。

4.priority_queue 内元素优先级的设置

如何定义优先队列内元素的优先级是运用好优先队列的关键。

YANZHISHI

（1）基本数据类型的优先级设置

此处指的基本数据类型就是 int 型、double 型、char 型等可以直接使用的数据类型，优先队列对它们的优先级设置一般是数字大的优先级越高，因此队首元素就是优先队列内元素最大的那个（如果 char 型，则是字典序最大的）。对基本数据类型来说，下面两种优先队列的定义是等价的（以 int 型为例，注意最后两个>之间有一个空格）：

```
priority_queue<int> q;
priority_queue<int, vector<int>, less<int>> q;
```

可以发现，第二种定义方式的尖括号内多出了两个参数：一个是 vector<int>，另一个是 less<int>。其中 vector<int>（也就是第二个参数）填写的是来承载底层数据结构堆(heap)的容器，如果第一个参数是 double 型或 char 型，则此处只需要填写 vector<double>或 vector<char>；而第三个参数 less<int>则是对第一个参数的比较类，**less<int>表示数字大的优先级越大，而 greater<int>表示数字小的优先级越大。**

因此，如果想让优先队列总是把最小的元素放在队首，只需进行如下定义：

```
priority_queue<int, vector<int>, greater<int>>q;
```

哈夫曼树

[3531. 哈夫曼树_AcWing题库]

输入格式

第一行包含整数 N，表示叶子结点数量。

第二行包含 N 个整数，表示每个叶子结点的权值。

输出格式

输出一个整数，表示生成哈夫曼树的带权路径长度。

数据范围

$2 \le N \le 1000$，
叶子结点的权值范围 $[1, 100]$。

输入样例：

```
5
1 2 2 5 9
```

输出样例：

```
37
```

研学士 YANZHISHI

课后习题

3432. 二叉树

KY85 二叉树

5070. 前序遍历

3540. 二叉搜索树

KY179 搬水果

研学士 YANZHISHI

谢谢聆听！